Calendly Integration

1. Overview

1.1 Description

Calendly Integration connects **Calendly** (scheduling automation platform) with **Newo Platform**.

It enables:

- Checking real-time availability by calculating open slots from scheduled events
- Booking appointments via the Calendly Invitee API with automatic outbound call location
- Cancelling existing appointments with reason tracking
- OAuth 2.0 authentication with automatic token refresh
- Auto-discovery and selection of event types from the Calendly account
- Auto-configuration of ConvoAgent booking/cancellation scenarios on canvas

This integration is designed for **businesses and service providers** who need **automated appointment scheduling through a conversational AI agent** (voice or chat), with Calendly as their calendar backend.

1.2 Use Cases

- When a client asks about available slots → Check Calendly scheduled events and return open time slots for the configured window (up to 7 days)
- When a client wants to book an appointment → Collect customer info (name, email, phone), create a Calendly invitee with outbound call location
- When a client wants to cancel an appointment → Cancel the scheduled event in Calendly with a cancellation reason
- When a conversation starts → Optionally auto-check availability and inject slots into the agent's context
- When the project is published → Automatically set up OAuth, fetch event types, inject booking schemas

1.3 Supported Features

Feature	Supported	Notes
Check Availability	✅	Configurable window (default: 5 days, max 7 per API limit)
Book Appointment	✅	With outbound call location and SMS reminders
Cancel Appointment	✅	With cancellation reason
OAuth 2.0 Authentication	✅	With automatic token refresh on 401 (up to 3 retries)
Event Type Auto-Discovery	✅	Auto-populates enum from active event types
Silent Availability Preload	✅	Optional, on session start
Canvas Auto-Setup	✅	Injects booking and cancellation scenarios with intents
Configurable Slot Duration	✅	Default: 30 minutes
Timezone Handling	✅	Converts between customer timezone and UTC

Reschedule Appointment	❌	Not implemented
Multi-Event-Type Support	❌	One event type per instance
Personal Access Token (PAT)	❌	OAuth 2.0 only

2. Requirements

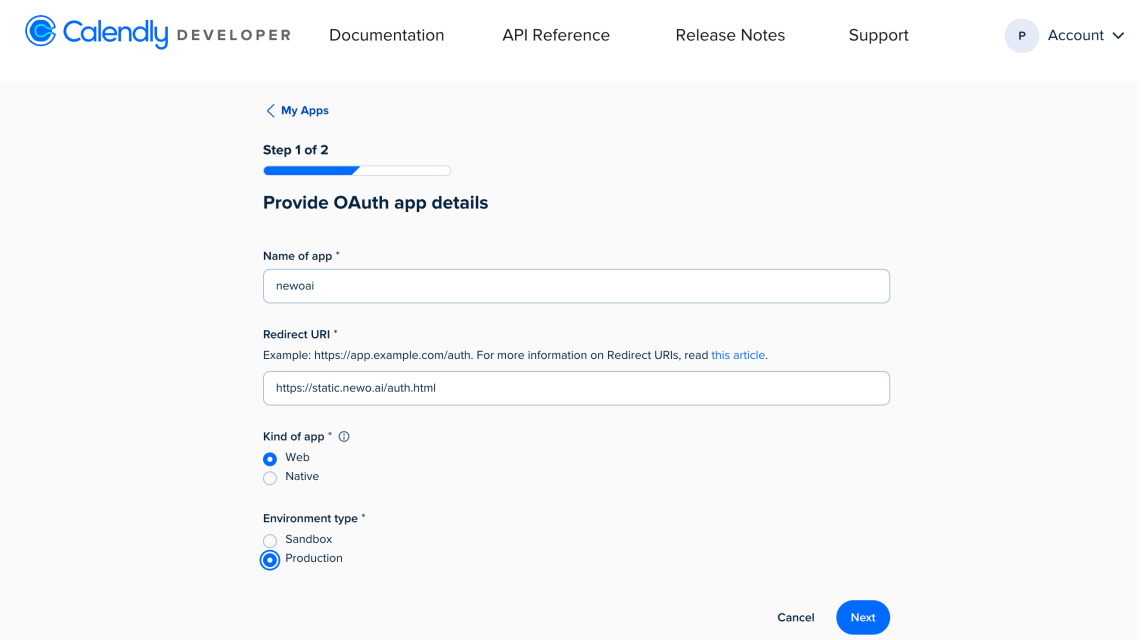
Before installation:

- Active **Calendly** account (Professional plan or higher for API access)
- **Calendly OAuth Application** registered in the [Developer Portal]
- At least one **Event Type** configured with **Location = Phone Call (Outbound)**

3. How to Setup

3.1 Step 1 — Create an OAuth Application in Calendly

1. Go to the [Calendly Developer Portal] and create a developer account if you don't have one
2. Navigate to **My Apps** and click **Create New App**
3. **Step 1 of 2 — Provide OAuth app details:**
   - **Name of app**: Enter a name (e.g., `newoai`)
   - **Redirect URI**: Enter `https://static.newo.ai/auth.html`
   - **Kind of app**: Select **Web**
   - **Environment type**: Select **Production**
   - Click **Next**



4. **Step 2 of 2 — Choose scopes:**
   - Enable **all scopes** in the following categories:

    - **Scheduling** (10/10): `availability:read`, `availability:write`, `event_types:read`, `event_types:write`, `locations:read`, `routing_forms:read`, `shares:write`, `scheduled_events:read`, `scheduled_events:write`, `scheduling_links:write`
    - **User management** (4/4): `groups:read`, `organizations:read`, `organizations:write`, `users:read`
    - **Security & Compliance** (3/3): `activity_log:read`, `data_compliance:write`, `outgoing_communications:read`
    - **Webhooks** (2/2): `webhooks:read`, `webhooks:write`
  - Click **Create**

Calendly DEVELOPER Documentation API Reference Release Notes Support

My Apps

Step 2 of 2

Choose scopes

Scopes define the permissions your app has to access data or perform actions in Calendly. When someone installs your app, they'll need to approve the scopes it requests.

Collapse all

Scheduling (10/10)

availability:read	Retrieve user availability and event-type availability.
availability:write	Update event type availability.
event_types:read	Retrieve event type details and available times.
event_types:write	Create or update event types.
locations:read	Retrieve configured meeting locations.
routing_forms:read	Retrieve routing forms and submissions.
shares:write	Create and customize a single-use scheduling link from an existing event type.
scheduled_events:read	Retrieve scheduled events and event invitee information.
scheduled_events:write	Create event invitees, cancel events, or mark invitees as no-show.
scheduling_links:write	Create a single-use scheduling link from an existing event type without any customization.

User management (4/4)

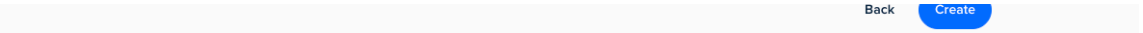
groups:read	Retrieve group details and relationships.
organizations:read	Retrieve organization data, memberships, and invitations.
organizations:write	Invite or remove users from an organization.
users:read	Retrieve user information.

Security & Compliance (3/3)

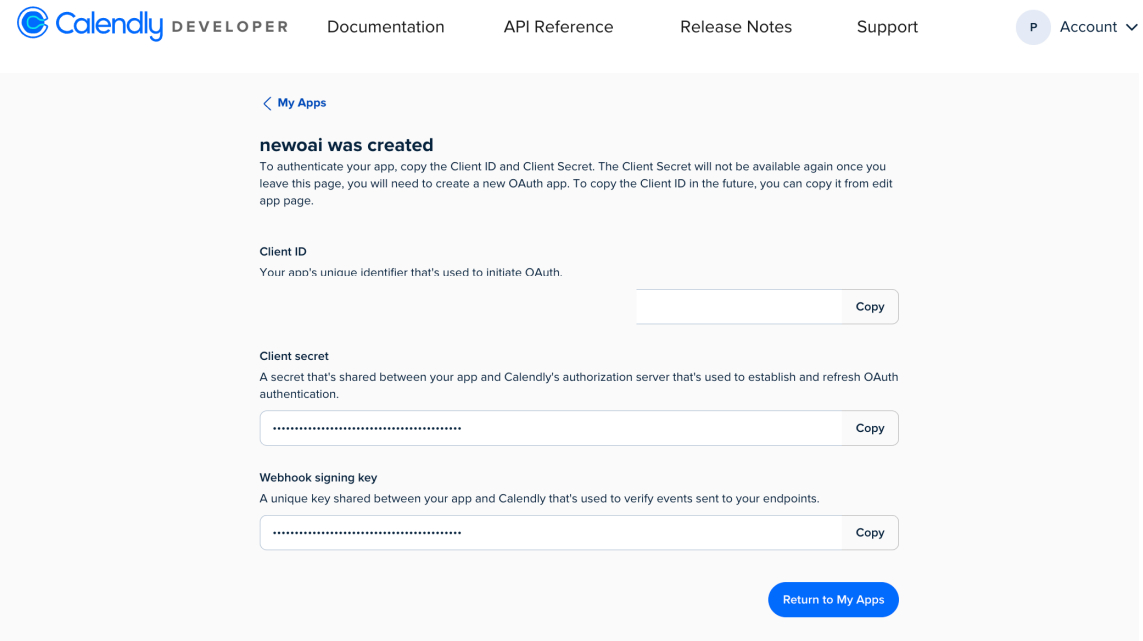
activity_log:read	View organization activity.
data_compliance:write	Delete invitee or event data.
outgoing_communications:read	Retrieve a list of outgoing SMS and email communications.

Webhooks (2/2)

webhooks:read	Retrieve webhook subscriptions and sample payloads.
webhooks:write	Create or delete webhook subscriptions.



5. Copy the **Client ID** and **Client Secret** — store them securely



> ***Important:*** *The Client Secret will not be available again once you leave this page. Save it immediately. To copy the Client ID later, you can find it on the edit app page.*

3.2 Step 2 — Configure Event Type in Calendly

The integration books appointments using **outbound phone calls**. You must configure at least one Event Type with the correct location.

1. Go to **Calendly** → **Event Types**
2. Select an existing event type or create a new one
3. In the event settings, set **Location** to **Phone Call** (Outbound — Calendly calls the invitee)

Booking page options

/30min • 30 min increments • auto time zone

Invitee form

Invitee details

Collect invitees first name and email, or full name and email

Name, Email

☐ Autofill Invitee Name, Email, and Text Reminder Phone Number (when applicable) from prior bookings

Invitee guests

Allow invitees to add guests

Edit location question

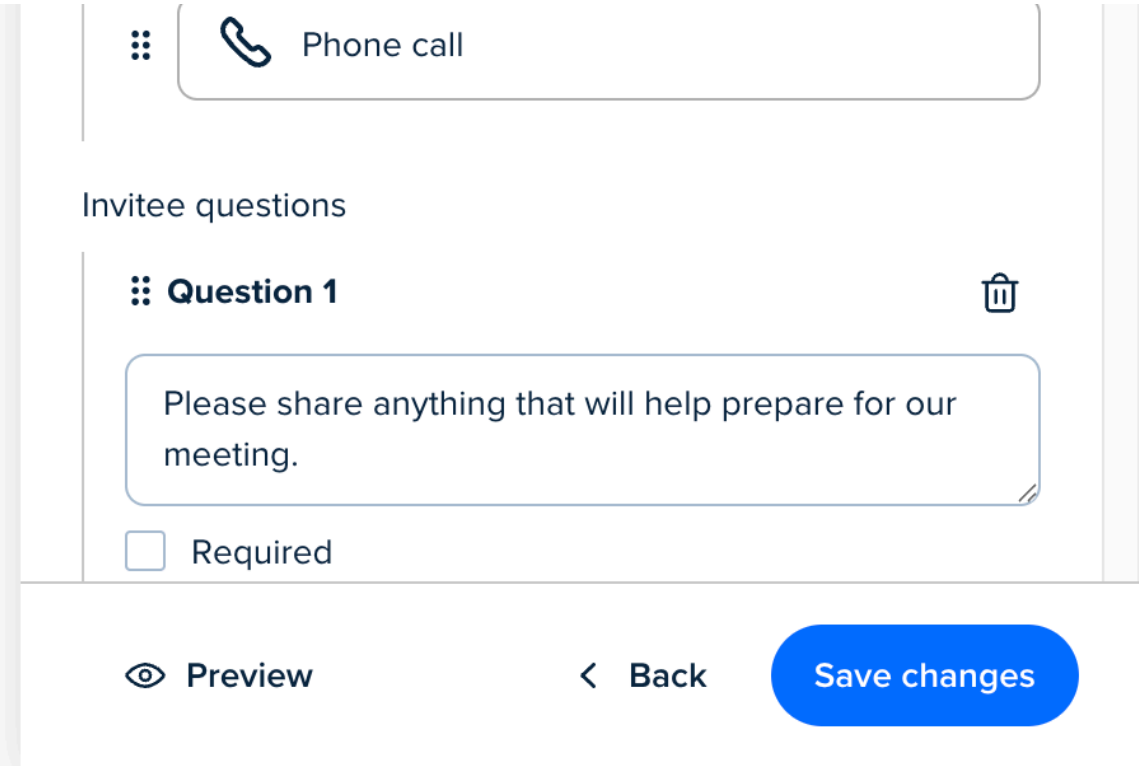
Customize the location question in your booking form. The location options are determined in the event details section.

Location label

Location

Location options

Google Meet



4. Configure your preferred **duration** (this maps to `calendly_duration` attribute, default: 30 min)
5. Save the event type

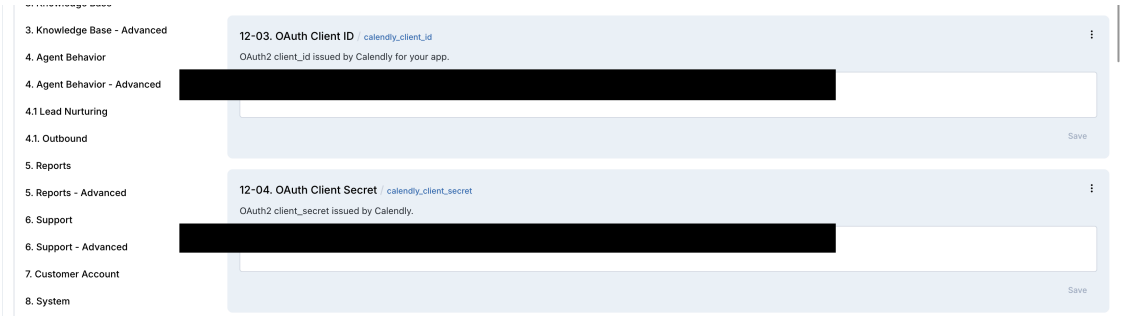
> ***Why Phone Call?*** *The integration sets* `location.kind = "outbound_call"` *in booking requests. Without a matching Phone Call location on the event type, Calendly will reject the booking.*

3.3 Step 3 — Connect in Newo Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes** (group: **12. Calendly Settings**):

Attribute	Required	Description
`calendly_client_id`	✅	OAuth2 Client ID from Calendly Developer Portal
`calendly_client_secret`	✅	OAuth2 Client Secret from Calendly Developer Portal
`calendly_redirect_uri`	❌	OAuth2 redirect URI (default: `https://static.newo.ai/auth.html`)
`calendly_base_url`	❌	API base URL (default: `https://api.calendly.com`)
`calendly_auth_url`	❌	Auth server URL (default: `https://auth.calendly.com`). Hidden.
`calendly_event_type_uri`	❌	Event type for booking. Auto-populated from API after OAuth setup.

`calendly_show_for_days`	❌	Days into future to show slots (default: `5`, max: 7)
`calendly_duration`	❌	Slot duration in minutes (default: `30`). Hidden.
`calendly_enable_slot_check`	❌	Enable availability checking (default: `True`)
`calendly_enable_booking`	❌	Enable booking capability (default: `True`)
`calendly_enable_cancellation`	❌	Enable cancellation capability (default: `True`)
`calendly_check_availability_on_conversation_start`	❌	Auto-check availability on session start (default: `False`)
`calendly_setup_scenarios`	❌	Auto-add booking/cancellation scenarios to canvas (default: `True`)
`calendly_override_agent_attributes`	❌	Override ConvoAgent attributes on setup (default: `True`)

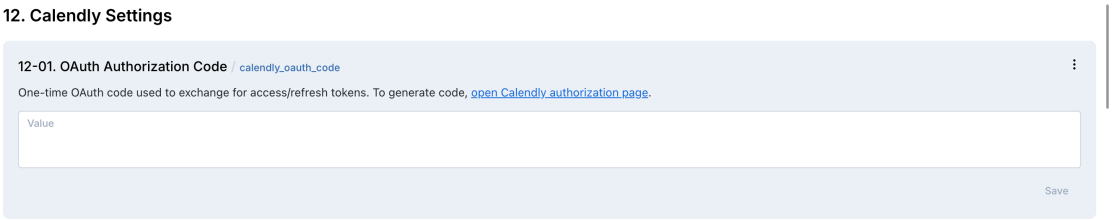


3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
   - Creates HTTP connectors (`calendly_connector` , `calendly_token_connector`)
   - Injects booking, availability, and cancellation schemas into ConvoAgent
   - Registers booking, availability, and cancellation tools with ConvoAgent
   - Adds pre-built booking and cancellation scenarios to the canvas (if enabled)

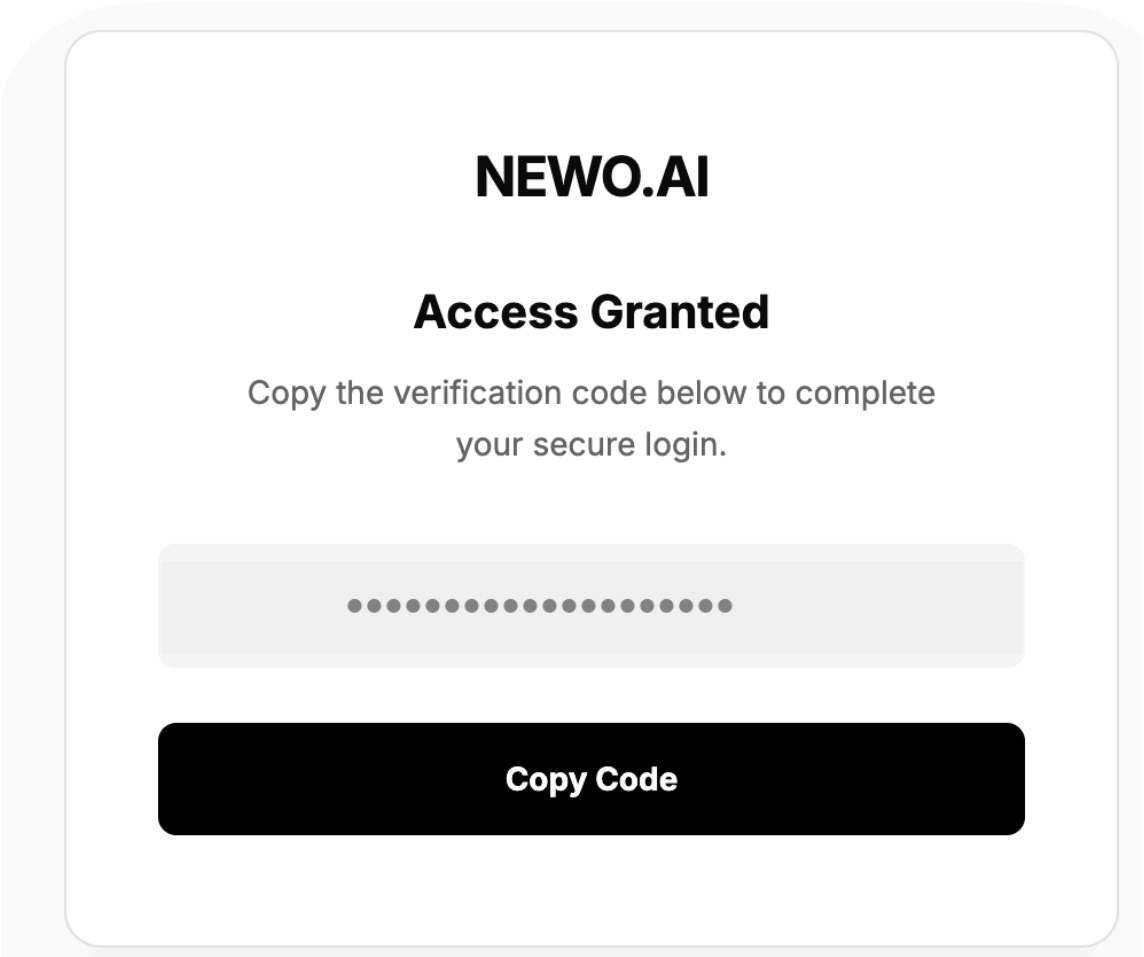
3.4 Step 4 — Authorize Calendly via OAuth

After publishing, you need to complete the OAuth authorization to connect your Calendly account.

1. In the **12. Calendly Settings** section, find the attribute **12-01. OAuth Authorization Code** (`calendly_oauth_code`)
2. In the description, click the **"open Calendly authorization page"** link



3. A Calendly authorization page will open. Approve the access request.
4. After approval, you will be redirected to the **NEWO.AI "Access Granted"** page showing a verification code
5. Click **Copy Code** to copy the authorization code



6. Go back to the Newo Platform and paste the copied code into the `calendly_oauth_code` attribute field
7. **IMPORTANT: Click the `Save` button on the attribute card first!** Do not skip this step — the code must be saved before publishing.
8. Click **Publish All**
9. **Reload the page in your browser** (F5 / Cmd+R) after publishing is complete
10. The `calendly_oauth_code` field should now be **empty** — this means the code was successfully exchanged for access/refresh tokens

> ***Note:*** *The OAuth code is one-time use. After a successful token exchange, the field is cleared automatically. If the field is not cleared after reloading, the code may have expired — repeat the authorization process from step 2.*

3.5 Step 5 — Select Event Type

After successful OAuth authorization, the integration fetches your active event types from Calendly.

1. Refresh the page and navigate to the attribute **`calendly_event_type_uri`**
2. A dropdown will appear with your active Calendly event types
3. **Select the event type** you want the agent to use for bookings (must have Location = Phone Call configured in Step 2)
4. Click **Save**, then click **Publish All**

> ***Important:*** *If the dropdown is empty, ensure the OAuth authorization (Step 4) was completed successfully. If you have only one active event type, it will be auto-selected.*

4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

4.1 How the Integration Works

The integration operates based on **Triggers and Actions** via the event-driven system.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Calendly

Available Triggers

Trigger	Event	Description
Check Availability	`get_available_slots`	When the agent needs to look up open time slots
Book Appointment	`book_slot`	When the agent confirms a booking
Cancel Appointment	`convo_agent_cancel_booking`	When the agent processes a cancellation
Session Start	`session_started`	Optionally preloads availability into agent context
Setup	`project_publish_finish`	Initializes integration on publish
Canvas Built	`gm_workflow_builder_canvas_built`	Injects Calendly scenarios into canvas

Available Actions

- **Get Scheduled Events** — Fetches active events for availability calculation (`GET /scheduled_events`)
- **Create Invitee** — Books an appointment with outbound call location (`POST /invitees`)
- **Cancel Event** — Cancels a scheduled event with reason (`POST /scheduled_events/{id}/cancellation`)
- **Get Event Types** — Discovers available event types during setup (`GET /event_types`)

- **Refresh Token** — Refreshes OAuth2 access token on 401 (`POST /oauth/token`)

4.2 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Calendly Integration
    participant API as Calendly API
    participant Auth as Calendly Auth

    Note over I: Setup (on publish)
    A->>I: project_publish_finish
    I->>I: Create connectors + attributes
    I->>Auth: POST /oauth/token (auth code)
    Auth-->>I: access_token, refresh_token
    I->>API: GET /event_types
    API-->>I: Active event types
    I->>A: Tools registered + schemas injected

    Note over U,API: Availability Check
    U->>A: "Any openings this week?"
    A->>I: get_available_slots (date, time)
    I->>API: GET /scheduled_events (date range)
    API-->>I: Booked events
    I->>I: Calculate open slots (subtract booked from window)
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book 2:30 PM please"
    A->>I: book_slot (customerInfo, dateTime)
    I->>I: Convert local time to UTC
    I->>API: POST /invitees (event_type, start_time, invitee)
    API-->>I: Scheduled event URI
    I->>I: Extract booking_id from URI
    I->>A: result_success (booking_id)
    A->>U: "Your appointment is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking (booking_id)
    I->>API: POST /scheduled_events/{id}/cancellation
    API-->>I: Cancellation confirmed
    I->>A: result_success (cancelled)
    A->>U: "Your appointment has been cancelled"

    Note over I,Auth: Token Refresh (on 401)
    I->>Auth: POST /oauth/token (refresh_token)
```

```
    Auth-->>I: New access_token
    I->>I: Retry original request
```

SetupFlow

```
flowchart TD
    E["project_publish_finish"] --> INIT["_init()<br>Create attributes, connectors,
<br>superagent attributes, persona"]
    INIT --> SCHEMA["_injectSchemaAttributes()<br>Availability + Booking +
<br>Cancellation schemas"]
    SCHEMA --> CODE{"OAuth code<br>present?"}
    CODE -->|"No"| SHOW["Show OAuth URL<br>to user"]
    CODE -->|"Yes"| TOKEN["_getTokens()<br>POST /oauth/token"]
    TOKEN --> SAVE["_getTokensSuccess()<br>Save access_token, refresh_token,
<br>owner_uri, organization_uri"]
    SAVE --> EVENTS["_getEventTypes()<br>GET /event_types?user={owner_uri}"]
    EVENTS --> POPULATE["_getEventTypesSuccess()<br>Populate enum with<br>active
event types"]
    POPULATE --> AUTO{"Only one<br>event type?"}
    AUTO -->|"Yes"| SELECT["Auto-select<br>event type"]
    AUTO -->|"No"| DONE["Done — user selects<br>from dropdown"]
```

AvailabilityFlow

```
flowchart TD
    E["get_available_slots<br>or session_started"] --> GATE{"Slot
check<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| EX["Extract: date, time<br>from payload"]
    EX --> VAL{"base_url, date, time,<br>owner_uri present?"}
    VAL -->|"No"| ERR["_checkAvailabilityError()"]
    VAL -->|"Yes"| UTC["Convert date/time<br>to UTC"]
    UTC --> RANGE["Calculate window:<br>start → start + show_for_days"]
    RANGE --> API["GET /scheduled_events<br>{min_start_time, max_start_time,
<br>status=active, user=owner_uri}"]
    API --> OK{"HTTP OK?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| PARSE["Parse booked events<br>Convert to customer TZ"]
    PARSE --> CALC["_getAvailableTimeSkill()<br>Generate all slots in
window<br>Subtract booked events"]
    CALC --> GROUP["Group slots by date<br>Format as HH:MM AM/PM"]
    GROUP --> OUT["result_success<br>{availability: [{location: date,
<br>availableTimeSlots: [...]}]}"]
```

BookingFlow

```
flowchart TD
    E["book_slot"] --> GATE{"Booking<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
```

```
    GATE -->|"Yes"| EX["Extract: bookingDateTime,<br>customerInfo from payload"]
    EX --> PERSONA["Read persona attributes:<br>full_name, email, phone"]
    PERSONA --> VAL{"Required fields<br>present?"}
    VAL -->|"No"| ERR["_executeResultError()"]
    VAL -->|"Yes"| UTC["Convert bookingDateTime<br>to UTC"]
    UTC --> BUILD["Build invitee payload:<br>event_type, start_time,<br>name, email,
phone,<br>location: outbound_call"]
    BUILD --> API["POST /invitees"]
    API --> OK{"HTTP OK?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| ID["Extract booking_id<br>from resource.event URI"]
    ID --> OUT["result_success<br>{booking_id}"]
```

CancellationFlow

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{"Cancellation<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| BID{"booking_id<br>in payload?"}
    BID -->|"No"| ERR["_cancelAppointmentError()<br>Escalate to manager"]
    BID -->|"Yes"| API["POST /scheduled_events/{id}/cancellation<br>{reason:
'Cancelled by Newo.AI agent'}"]
    API --> OK{"HTTP OK?"}
    OK -->|"No"| ERR2["ResultError"]
    OK -->|"Yes"| CHECK{"response.resource<br>.reason exists?"}
    CHECK -->|"Yes"| OUT["result_success<br>{success: true}"]
    CHECK -->|"No"| FAIL["result_success<br>{success: false}"]
```

UtilsFlow (Token Refresh)

```
flowchart TD
    E["Any HTTP request"] --> EXEC["_executeRequest()<br>Add Bearer token header"]
    EXEC --> SEND["SendCommand → HTTP connector"]
    SEND --> RES{"HTTP<br>response"}
    RES -->|"200 OK"| SUCCESS["Route to runSkill<br>via calendly_result_success"]
    RES -->|"401"| AUTH{"Auth type?"}
    AUTH -->|"OAuth2"| REFRESH["_refreshToken()<br>POST /oauth/token<br>{grant_type:
refresh_token}"]
    AUTH -->|"Token"| ERR["calendly_result_error"]
    REFRESH --> TOKEN{"Refresh<br>successful?"}
    TOKEN -->|"Yes"| SAVE["_saveToken()<br>Update access_token"]
    TOKEN -->|"No"| ERR
    SAVE --> RETRY{"Attempts<br>< 3?"}
    RETRY -->|"Yes"| EXEC
    RETRY -->|"No"| ERR
    RES -->|"Other error"| ERR
```

CanvasSetupFlow

```
flowchart TD
    E["gm_workflow_builder_canvas_built"] --> GATE{"Canvas setup<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| LIB["SetupLibrarySkill()<br>Add items to canvas library"]
    LIB --> DELETE["Delete generic scenarios:<br>home_service_appointment,
<br>appointment_scheduling,<br>closing_via_appointment,<br>cancellation"]
    DELETE --> ADD["Add Calendly scenarios:<br>speed_to_lead_scenario,
<br>cancellation_via_agent_scenario,<br>common_schedule_appointment"]
    ADD --> INTENT["Add intent types:<br>cancellation_via_agent_intent,
<br>common_appointment_agent"]
    INTENT --> DONE["Canvas configured"]
```

4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel) by interacting with the agent. API responses are logged via the integration's HTTP connector.

4.4 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify:
    - OAuth authorization URL appears — click it to authorize
    - After OAuth, `calendly_event_type_uri` dropdown shows your event types
    - Select the desired event type and re-publish
2. **Availability**: Ask the agent "What slots are available this week?" — verify:
    - Slots are returned grouped by date
    - Times match your Calendly calendar (booked times excluded)
    - Times are in your business timezone
3. **Booking**: Complete a booking conversation with name, email, phone, and time — verify:
    - Appointment appears in your Calendly dashboard
    - Location shows as "Phone Call"
    - Invitee details (name, email, phone) are correct
4. **Cancellation**: Request cancellation of a booked appointment — verify:
    - Event status changed to "cancelled" in Calendly
    - Cancellation reason shows "Cancelled by Newo.AI agent"

If no action occurs:

- Ensure `calendly_client_id` and `calendly_client_secret` are set correctly
- Verify OAuth authorization was completed (access token should be populated)
- Confirm `calendly_event_type_uri` is selected (not empty)
- Check that the relevant feature flags are enabled (`calendly_enable_booking` , `calendly_enable_slot_check` , `calendly_enable_cancellation`)
- Verify the selected Event Type has **Location = Phone Call** in Calendly
- Check that `calendly_base_url` is correct (default: `https://api.calendly.com`)
- Re-publish after making any configuration changes

> *Note: This integration performs real operations in Calendly. Appointments created or cancelled through the agent are reflected in the live Calendly system.*

5. FAQ

Q: Does the integration import historical appointments? A: No. Only appointments created after activation are managed. The availability check reads existing scheduled events to calculate open slots, but does not import them.

Q: Can I connect multiple Calendly accounts? A: Each integration instance supports one OAuth connection and one event type. For multiple accounts or event types, create separate integration instances.

Q: Why must the Event Type location be "Phone Call"? A: The integration creates bookings with `location.kind = "outbound_call"`, which tells Calendly to call the invitee. If the Event Type doesn't have a Phone Call location configured, Calendly will reject the booking request.

Q: How is availability calculated? A: The integration fetches all active scheduled events from Calendly within the configured window (default: 5 days). It then generates all possible time slots based on the configured duration (default: 30 min) and removes any slots that overlap with existing events. The remaining slots are returned grouped by date.

Q: What happens if the OAuth token expires? A: The integration automatically detects 401 responses and refreshes the token using the stored refresh token. It retries the original request up to 3 times. If refresh fails, an error is reported to the agent.

Q: What data does the agent collect for booking? A: Required: **first name**, **email**, and **phone number** (with country code). Optional: last name. The phone number is also used as the outbound call location. The agent reads persona attributes first and only asks the user for missing information.

Q: What scenarios are added to the canvas? A: When `calendly_setup_scenarios` is enabled, the integration adds: a Speed-to-Lead outbound call scenario (for initial greeting and intent clarification), a Cancellation via Agent scenario (with required confirmation phrases), and a common appointment scheduling scenario. Generic appointment/cancellation scenarios are removed to avoid conflicts.